Table des matières

Cours 05 — Rendre le pipeline robuste 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — La porte d'entrée, `pipeline.py` · Tenir le pipeline 2
Étape 2 — Faire tourner le pipeline · Tenir le pipeline 3
Étape 3 — Le pipeline qui échoue · Tenir le pipeline 4
Étape 4 — Un test qui prouve que relancer ne double rien · Tenir le pipeline 5
Étape 5 — Versionner · Tenir le pipeline 7
À retenir 7

Cours 05 — Rendre le pipeline robuste

Cours 5/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici · Tenir le pipeline, ce qui traverse les trois étapes : le code, l'orchestration, la qualité	✓	✓

Ce que vous allez faire

Le bronze se lance par `duckdb -f`, la `RAW` par `python 01_de\charger.py` : deux commandes, qu'on tape dans le bon ordre, et dont personne ne lit le résultat sinon vous. Un pipeline robuste a une seule porte d'entrée : elle enchaîne les étapes, dit ce que chacune a fait, s'arrête à la première qui échoue et le dit par son code de sortie, que la machine lit. Vous écrivez `pipeline.py`, vous le faites échouer exprès, puis vous écrivez un test qui prouve, sans les données du café, que relancer le chargement ne double rien.

`pipeline.py <jour>` → le bronze, refait et contrôlé → la `RAW`, chargée jusqu'au jour → un code de sortie ; puis `pytest`, qui rejoue le chargement dans un projet jetable

Le code complet est dans `Codes_05_DE.md`, à ouvrir après avoir essayé.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, Python, `pytest` et Git ; la seconde pour le notebook, à l'étape 3 seulement. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
pytest --version
REM Attendu : pytest suivi de son numéro de version.
```

`pytest` s'est installé avec le reste de l'environnement `formations`.

Étape 1 — La porte d'entrée, `pipeline.py` · Tenir le pipeline

Le pipeline a deux étapes, dans cet ordre : le bronze, qui se refait en entier à partir des sources, puis la `RAW`, qui charge ce qui est arrivé jusqu'au jour donné. `meteo.py` n'en est pas : il demande Internet, et le mois figé n'a rien de neuf à demander à l'API.

Chaque étape est une fonction qui rend, en quelques mots, ce qu'elle a fait, ou qui lève une exception si elle échoue. La porte d'entrée les appelle l'une après l'autre, écrit leur réponse, et rend un code de sortie : 0 si tout est passé, 1 à la première étape en échec. C'est la seule chose qu'un orchestrateur lit avec certitude : un pipeline qui écrirait « échec » et rendrait 0 serait compté comme réussi.

Pour remplir	**L'outil**
réutiliser le chargeur du cours 03	`from charger import charger` : `charger.py` est à côté, dans `01_de`, et Python le trouve
lancer `bronze.sql` depuis Python	`duckdb.sql(texte)` exécute tout le texte du fichier, en mémoire, et `.fetchone()` rend la ligne de sa dernière requête, le contrôle
faire échouer une étape	`raise ValueError("…")`
attraper l'échec d'une étape	`try:` … `except Exception as erreur:`, et `erreur` porte le message
rendre le code de sortie	`sys.exit(code)`

Le contrôle de `bronze.sql`, que vous lisiez à l'œil au cours 02, devient une garde : si la source et le bronze n'ont pas autant de tickets, l'étape échoue, et la `RAW` ne se charge pas.

1.1 Créer le fichier, dans la première fenêtre.

```
notepad 01_de\pipeline.py
```


© 2026 iAlexMG. Tous droits réservés.

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

```
# 01_de/pipeline.py : la porte d'entrée du pipeline, qui enchaîne les étapes et rend un code de sortie
import sys
import duckdb
from charger import charger

def bronze(jusqua):
    # 1. le bronze, refait en entier ; son contrôle doit compter autant de tickets des deux côtés
    source, tickets = duckdb.sql(open("01_de/sql/bronze.sql", encoding="utf-8").read())._____()
    if source != tickets:
        raise ValueError(f"{source} tickets à la source, {tickets} dans le bronze")
    return f"{tickets} tickets rangés"

def raw(jusqua):
    # 2. la RAW, chargée de ce qui est arrivé jusqu'au jour donné
    with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:

        _____
        return f"{con.execute('SELECT COUNT(*) FROM journal').fetchone()[0]} fichiers au journal"

def lancer(jusqua):
    # 3. chaque étape tracée ; la première qui échoue arrête tout, et le code de sortie vaut 1
    for nom, etape in (("bronze", bronze), ("raw", raw)):
        try:
            print(f"{nom} : {etape(jusqua)}")
        except _____ as erreur:
            print(f"{nom} : échec, {erreur}")
            return _____
    return 0

if __name__ == "__main__":
    sys.exit(lancer(sys.argv[1]))
```

Enregistrer, puis fermer le Bloc-notes.

Étape 2 — Faire tourner le pipeline · Tenir le pipeline

2.1 Lancer le pipeline sur tout le mois, puis lire son code de sortie. `%ERRORLEVEL%` garde le code de la dernière commande.

```
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : 1527 tickets rangés, puis raw : 75 fichiers au journal.
```


© 2026 iAlexMG. Tous droits réservés.

```
echo %ERRORLEVEL%
REM Attendu : 0.
```

La `RAW` avait déjà tout le mois, depuis le cours 03 : le chargeur n'a rien chargé, et le journal compte toujours 75 fichiers. Le bronze, lui, s'est refait. Relancer le pipeline ne change rien : c'est ce qui permet de le relancer sans crainte après une panne.

2.2 Regarder le dépôt.

```
dir /b 01_de
REM Attendu : .gitignore, charger.py, meteo.py, pipeline.py, sql et __pycache__.
```

`__pycache__` est apparu : en important `charger.py`, Python a rangé sa version compilée à côté. Le `.gitignore` du cours 02 l'écarte déjà.

Étape 3 — Le pipeline qui échoue · Tenir le pipeline

Un pipeline se juge aussi à la façon dont il échoue. L'échec se provoque sans rien casser : le notebook, qui tient l'entrepôt, suffit, comme au cours 03.

3.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 05 ». Dans sa première cellule, attacher l'entrepôt en lecture seule, et l'exécuter.

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie.duckdb' AS entrepot (READ_ONLY);
-- Attendu : entrepot paraît sous Attached databases.
```

3.2 Lancer le pipeline pendant que le notebook tient l'entrepôt, dans la première fenêtre.

```
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : 1527 tickets rangés, puis raw : échec, IO Error: Cannot open file
REM "…\brulerie.duckdb": Le processus ne peut pas accéder au fichier…, et File is already open in
REM …\duckdb.exe.
echo %ERRORLEVEL%
REM Attendu : 1.
```

Le bronze est passé : il n'ouvre pas l'entrepôt. La `RAW` a échoué, son message dit pourquoi, et le code de sortie dit à la machine qu'il ne faut pas continuer.

3.3 Rendre l'entrepôt, puis relancer. Dans le notebook, une cellule neuve :

```
DETACH DATABASE IF EXISTS entrepot;
```

Puis, dans la première fenêtre :

```
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : 1527 tickets rangés, puis raw : 75 fichiers au journal.
```


© 2026 iAlexMG. Tous droits réservés.

```
echo %ERRORLEVEL%
REM Attendu : 0.
```

3.4 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 4 — Un test qui prouve que relancer ne double rien · Tenir le pipeline

Le pipeline tourne sur les données du café ; un test, lui, se monte son propre petit monde, avec juste ce qu'il faut pour prouver une seule chose. `pytest` cherche les fichiers `test_*.py`, y lance chaque fonction `test_…`, et compte comme échoué tout `assert` faux.

Pour remplir	**L'outil**
un dossier jetable, neuf à chaque test	le paramètre `tmp_path`, que `pytest` crée et passe à la fonction
se placer dans ce dossier pendant le test	`monkeypatch.chdir(tmp_path)` : `charger` y cherche `sources/`, et le dossier courant revient à la fin
un entrepôt qui ne laisse rien sur le disque	`duckdb.connect()`, sans nom de fichier : en mémoire
vérifier	`assert condition` : le test échoue si elle est fausse

`pytest` lit sa configuration dans `pytest.ini` : où sont les tests, et où trouver le code qu'ils importent. Sans la ligne `pythonpath`, `from charger import charger` ne trouverait pas `charger.py`, qui n'est pas dans le dossier `tests`.

4.1 Écrire la configuration.

```
notepad 01_de\pytest.ini
```

Accepter la création, puis écrire :

```
# 01_de/pytest.ini : où pytest cherche les tests, et le code qu'ils importent
[pytest]
# 1. le dossier du dépôt, pour que « from charger import charger » trouve charger.py
pythonpath = ____
testpaths = tests
```

Enregistrer, puis fermer le Bloc-notes.

4.2 Écrire le test.

© 2026 iAlexMG. Tous droits réservés.

```
mkdir 01_de\tests
notepad 01_de\tests\test_charger.py
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

```
# 01_de/tests/test_charger.py : relancer le chargement ne double rien
import duckdb

from charger import charger

def test_relancer_ne_double_rien(tmp_path, monkeypatch):
    # 1. un projet jetable : deux exports de tickets, livrés le 1er et le 2 juin
    caisse = tmp_path / "sources" / "caisse" / "2025-06"
    caisse.mkdir(parents=True)
    for jour in ("01", "02"):
        (caisse / f"tickets_2025-06-{jour}_202506{jour}1707.csv").write_text(f"ticket_id\n{jour}\n")
    monkeypatch.chdir(____)
    con = duckdb.connect()
    # 2. charger le 1er juin : un seul fichier est arrivé
    charger(con, "2025-06-01")
    assert con.execute("SELECT COUNT(*) FROM raw_tickets").fetchone()[0] == ____
    # 3. charger le mois, puis le relancer : deux fichiers au journal, deux tickets, rien de doublé
    charger(con, "2025-06-30")
    charger(con, "2025-06-30")
    assert con.execute("SELECT COUNT(*) FROM journal").fetchone()[0] == 2
    assert ____
```

Enregistrer, puis fermer le Bloc-notes.

4.3 Lancer le test. `pytest 01_de` lit `01_de\pytest.ini`, puis lance les tests du dossier `tests`.

```
pytest 01_de
REM Attendu : rootdir: …\01_de, configfile: pytest.ini, collected 1 item, puis 1 passed, en une
REM fraction de seconde.
```

Si `pytest` s'arrête sur	La cause, et le geste
`ModuleNotFoundError: No module named 'charger'`, puis `Interrupted: 1 error during collection`	`pythonpath` manque à `pytest.ini`, ou le fichier n'est pas dans `01_de`
`collected 0 items`	le fichier ou la fonction ne commence pas par `test_`

© 2026 iAlexMG. Tous droits réservés.

Si `pytest` s'arrête sur	La cause, et le geste
`1 failed`, et la ligne de l'`assert` en faute	le chargeur a doublé, ou n'a pas tout pris : `pytest` montre les deux valeurs comparées

Le test ne lit pas une ligne des données du café : il fabrique deux exports de quelques octets, charge, relance, compte. Le jour où quelqu'un retouche `charger.py`, `pytest` dit en une seconde si la relance double encore rien.

Étape 5 — Versionner · Tenir le pipeline

`pytest` a laissé un dossier de cache dans le dépôt, `.pytest_cache`, et un `__pycache__` dans `tests` : ni l'un ni l'autre n'est du code.

5.1 Écarter le cache de `pytest`.

```
notepad 01_de\.gitignore
```

Ajouter ces deux lignes au bout du fichier :

```
# Le cache de pytest
.pytest_cache/
```

Enregistrer, puis fermer le Bloc-notes.

5.2 Valider la porte d'entrée, le test et sa configuration.

```
git -C 01_de add .gitignore pipeline.py pytest.ini tests
git -C 01_de commit -m "Cours 05 : la porte d'entrée du pipeline, et un test"
REM Attendu : 4 files changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Un pipeline a une seule porte d'entrée : elle enchaîne les étapes dans l'ordre, dit ce que chacune a fait, et s'arrête à la première qui échoue.
- L'échec se dit par le code de sortie : 0 si tout est passé, 1 sinon ; c'est ce que la machine lit.
- Un test se monte son propre petit monde et prouve une seule chose : ici, que relancer le chargement ne double rien.

© 2026 iAlexMG. Tous droits réservés.